



Why The Free Monad Isn't Free


```
[error] Exception encountered  
[error] java.lang.StackOverflowError
```


**“Let’s just trampoline it and
add the Free Monad”**

“Let
a
t and
”



@kelleyrobinson





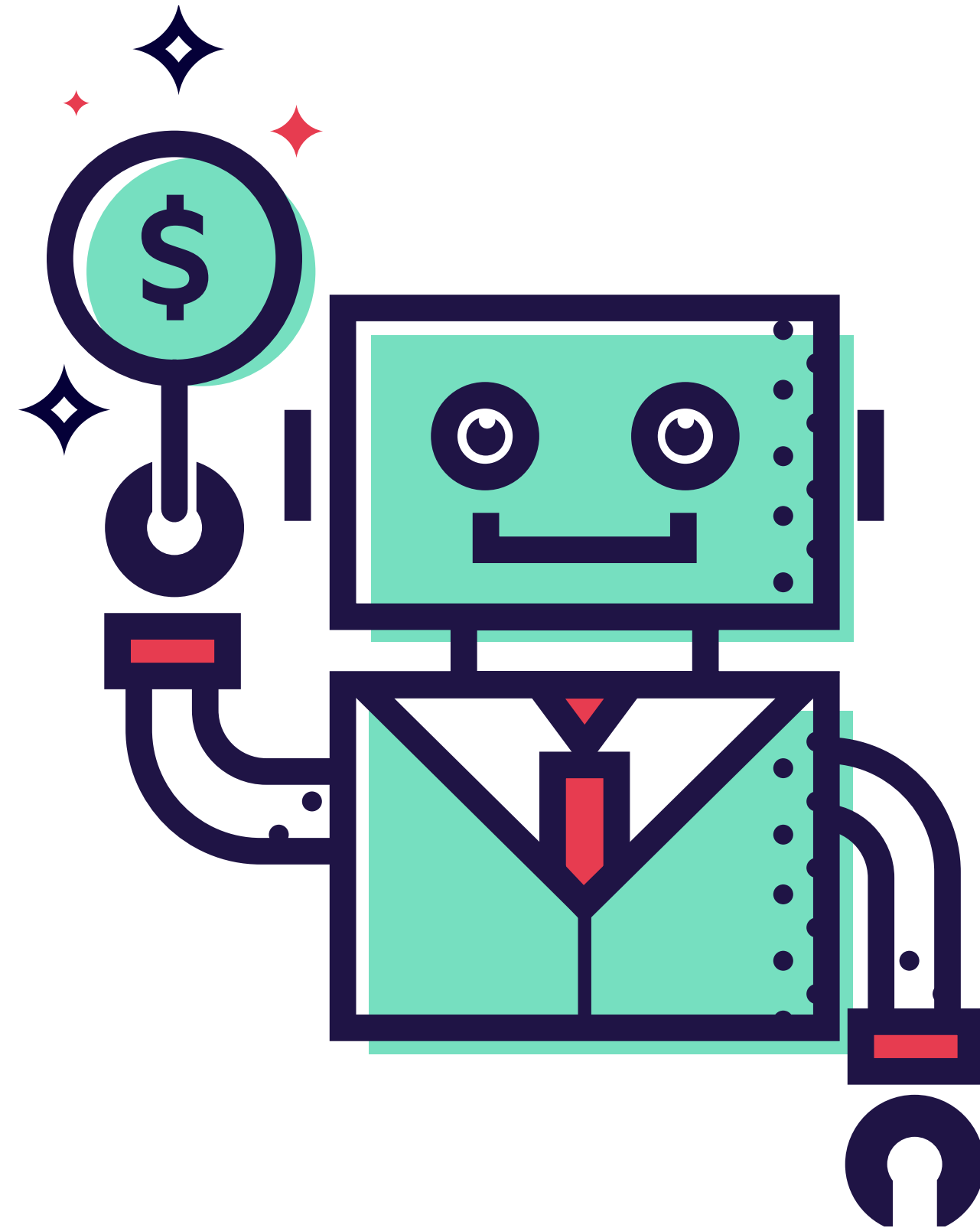
Why The Free Monad Isn't Free

Kelley Robinson

Data & Infrastructure Engineer

Sharethrough

@kelleyrobinson



- Monoids, Functors & Monads
- How to be “Free”
- Why & Why Not “Free”
- Alternatives
- Real World Applications

github.com/robinske/monad-examples



Richard Minerich

@rickasaurus



Follow

burritos are just tacoids in the category of enchiladafunctors

12:56 PM - 2 Mar 2016



385



442

Monoids


```
•  
  
trait Monoid[A] {  
  def append(a: A, b: A): A  
  def identity: A  
}
```


Monoids



Image credit: deluxebattery.com



@kelleyrobinson

Properties

Identity: "no-op" value

Associativity:
grouping doesn't matter

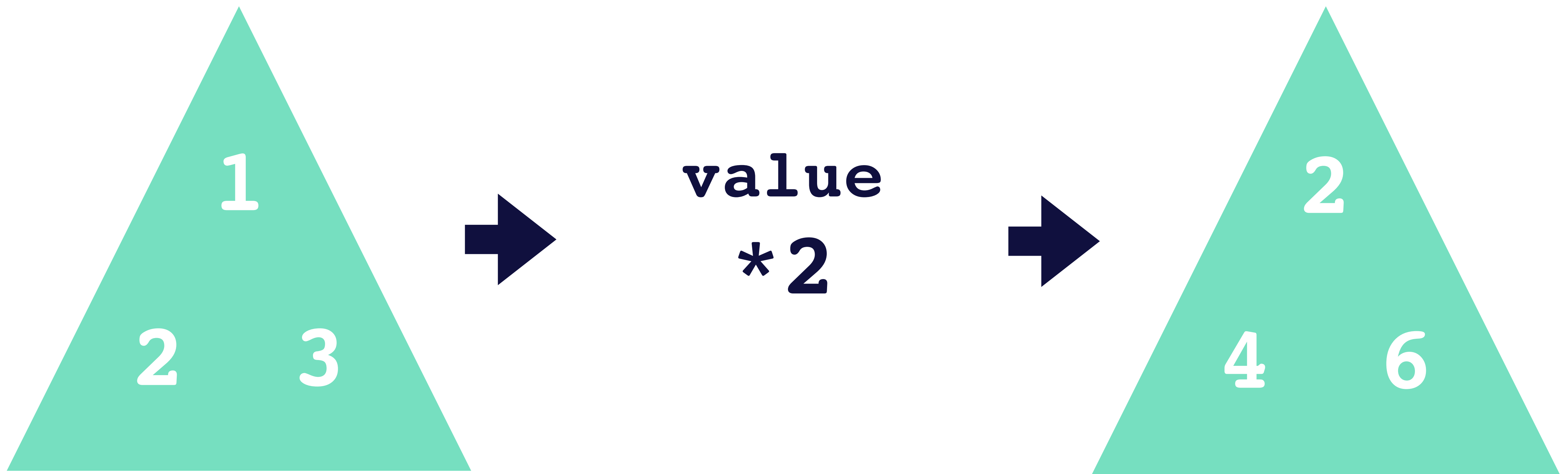

```
object StringConcat extends Monoid[String] {  
  def append(a: String, b: String): String = a + b  
  def identity: String = ""  
}
```

```
object IntegerAddition extends Monoid[Int] {  
  def append(a: Int, b: Int): Int = a + b  
  def identity: Int = 0  
}
```

Functors


```
trait Functor[F[_]] {  
  def map[A, B](a: F[A])(fn: A => B): F[B]  
}
```


map





@kelleyrobinson

Properties

Identity: "no-op" value

Composition:
grouping doesn't matter

Monads

"The term **monad is a bit
vacuous if you are not a
mathematician. An alternative
term is **computation builder**."**

```
trait Monad[M[_]] {  
  def pure[A](a: A): M[A]  
  def flatMap[A, B](a: M[A])  
    (fn: A => M[B]): M[B]  
}
```

```
trait Monad[M[_]] {  
  def pure[A](a: A): M[A]  
  def flatMap[A, B](a: M[A])(fn: A => M[B]): M[B]  
}
```



```
trait Monad[M[_]] {  
  
  def pure[A](a: A): M[A]  
  
  def flatMap[A, B](a: M[A])(fn: A => M[B]): M[B]  
  
  def map[A, B](a: M[A])(fn: A => B): M[B] = {  
    flatMap(a){ b: A => pure(fn(b)) }  
  }  
  
}
```

```
trait Monad[M[_]] {  
  
  def pure[A](a: A): M[A]  
  
  def flatMap[A, B](a: M[A])(fn: A => M[B]): M[B]  
  
  def map[A, B](a: M[A])(fn: A => B): M[B] = {  
    flatMap(a){ b: A => pure(fn(b)) }  
  }  
}
```

```

trait Monad[M[_]] {

  def flatMap[A, B](a: M[A])(fn: A => M[B]): M[B]

  def append[A, B, C]
    (f1: A => M[B], f2: B => M[C]): A => M[C] = {
    a: A =>
      val bs: M[B] = f1(a)
      val cs: M[C] = flatMap(bs) { b: B =>
        f2(b)
      }
      cs
    }
}

```



```
trait Monad[M[_]] {  
  
  def flatMap[A, B](a: M[A])(fn: A => M[B]): M[B]  
  
  def append[A, B, C]  
    (f1: A => M[B], f2: B => M[C]): A => M[C] = {  
    a: A =>  
      val bs: M[B] = f1(a)  
      val cs: M[C] = flatMap(bs) { b: B =>  
        f2(b)  
      }  
      cs  
    }  
  
}
```



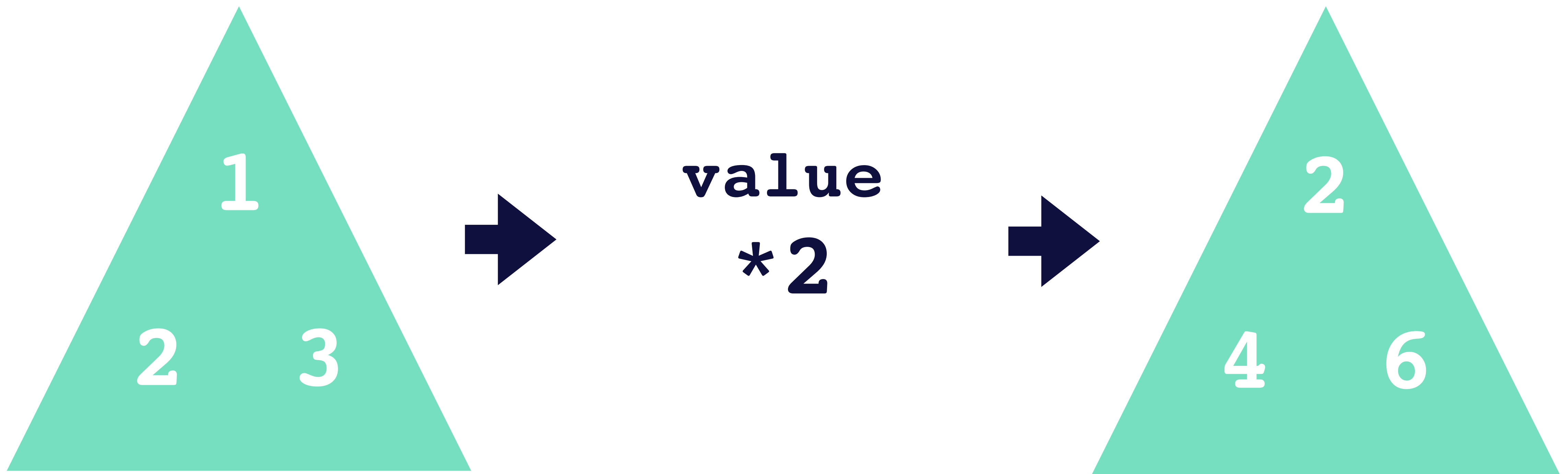
@kelleyrobinson

Properties

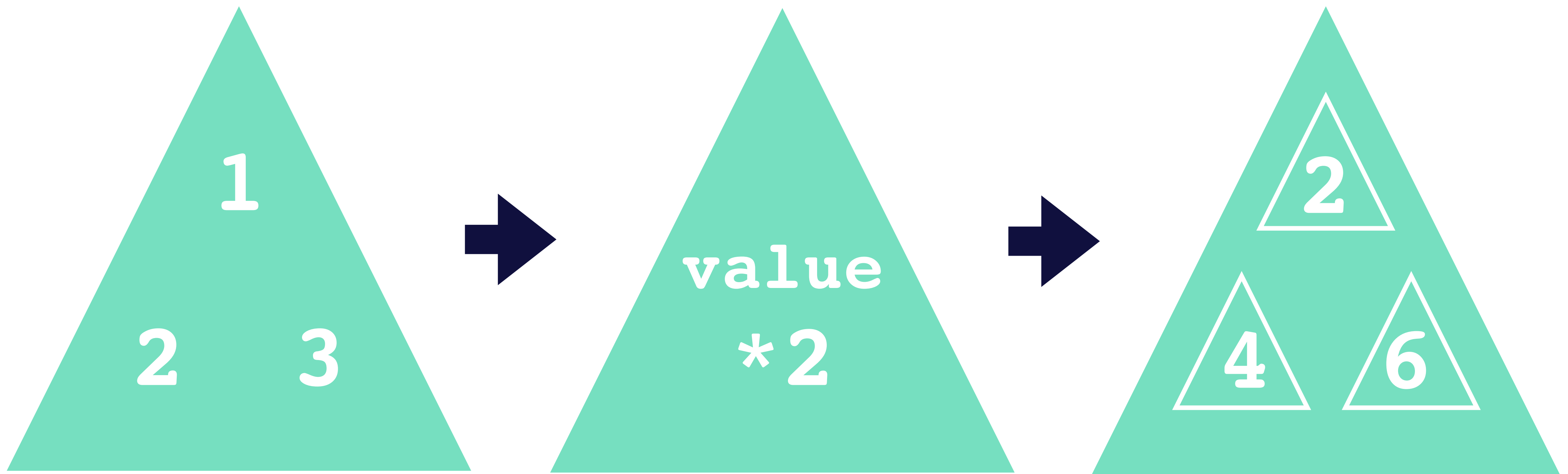
Identity: "no-op" value

Composition:
grouping doesn't matter

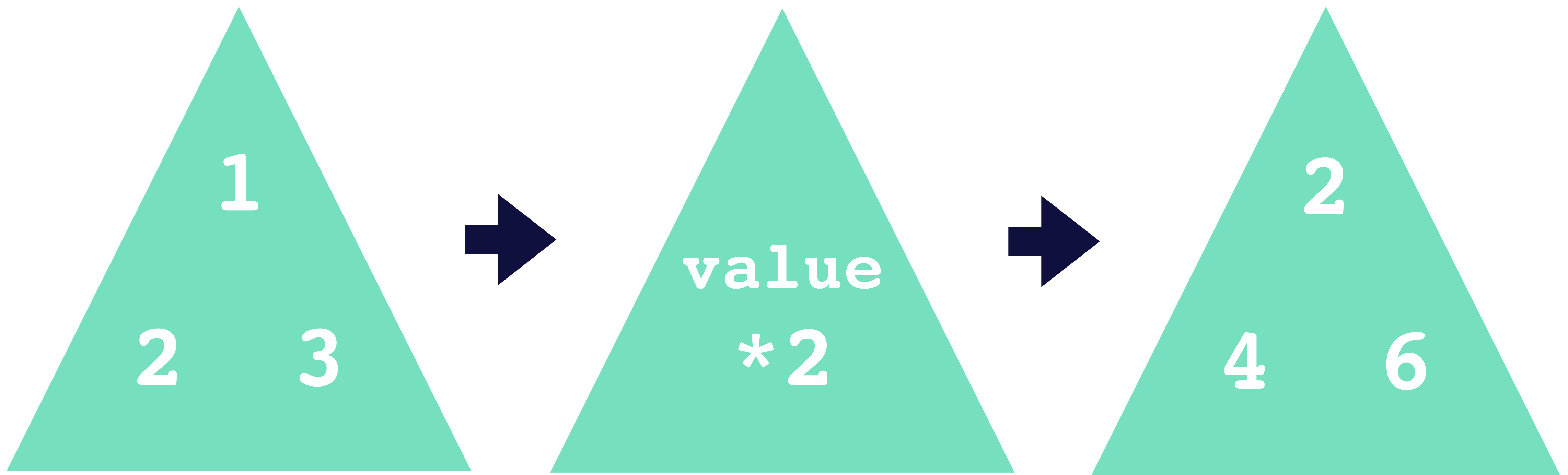
map (functor)

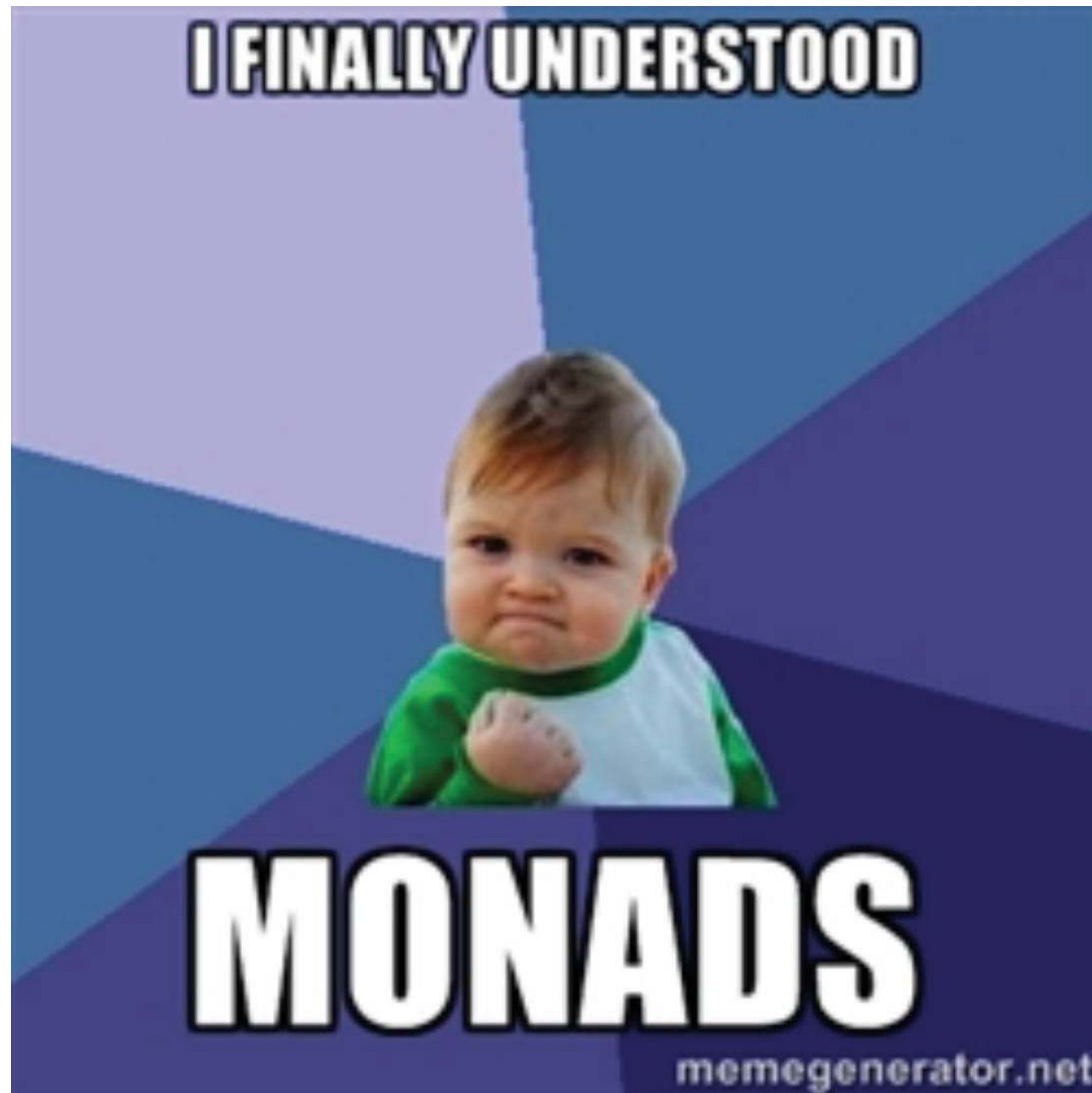


map



flatMap

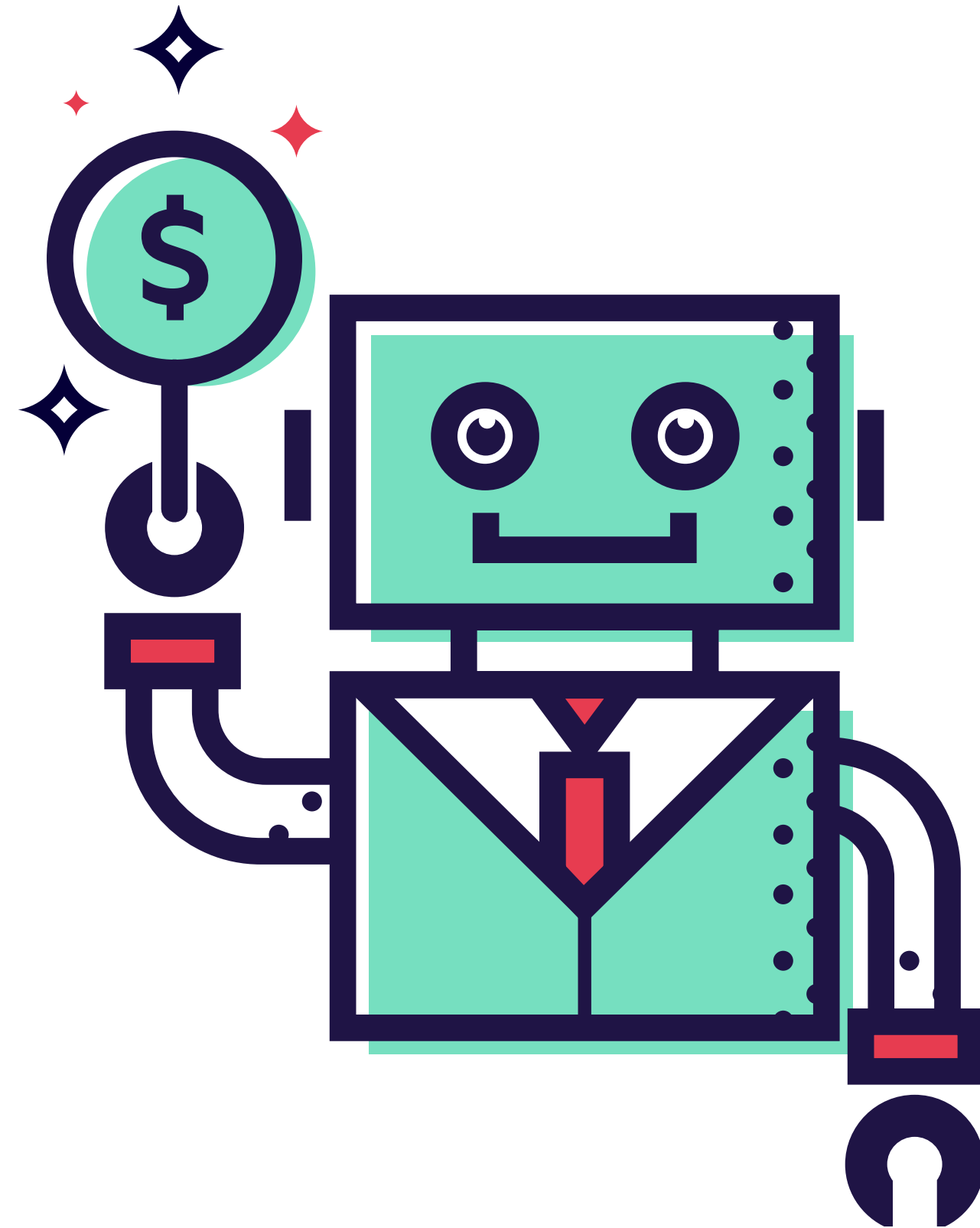




@kelleyrobinson

Compose functions
for **values in a
context**

Think: Lists, Futures



- Monoids, Functors & Monads
- How to be “Free”
- Why & Why Not “Free”
- Alternatives
- Real World Applications



*The word "free" is
used in the sense of
"unrestricted" rather
than "zero-cost"*

"Freedom not beer"



https://en.wikipedia.org/wiki/Gratis_versus_libre#/media/File:Galuel_RMS_-_free_as_free_speech,_not_as_free_beer.png

Free Monoids

```
•  
  
trait Monoid[A] {  
  def append(a: A, b: A): A  
  def identity: A  
}
```

Free Monoids

- Free from interpretation
- No lost input data when appending

@kelleyrobinson

// I'm free!

```
class ListConcat[A] extends Monoid[List[A]] {  
  def append(a: List[A], b: List[A]): List[A] =  
    a ++ b  
  
  def identity: List[A] = List.empty[A]  
}
```

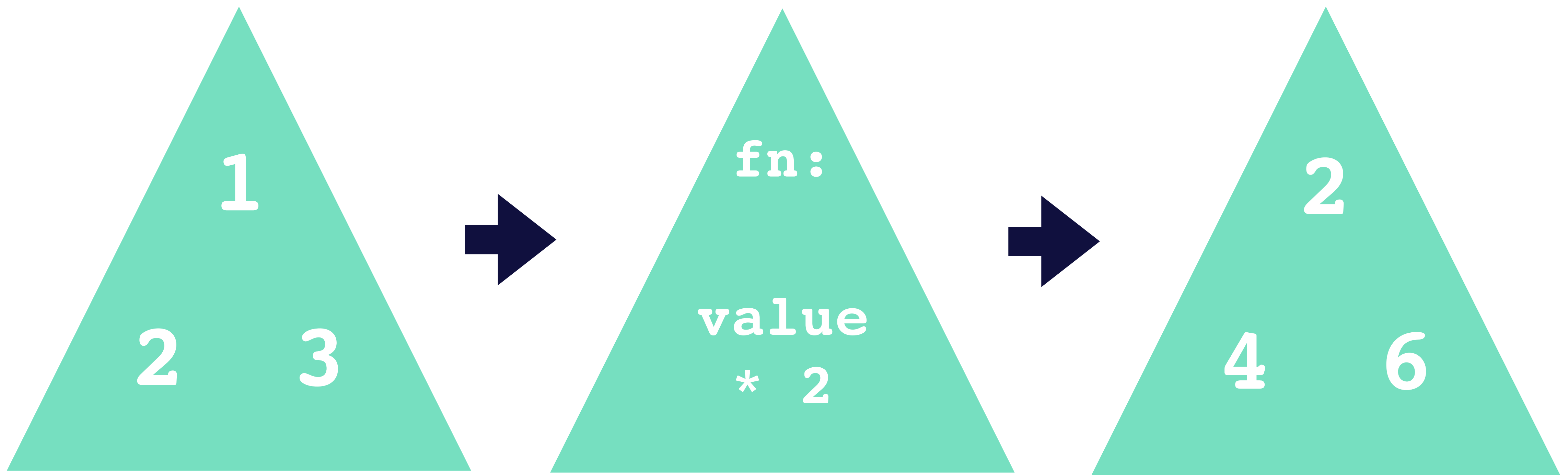
// I'm not free :(

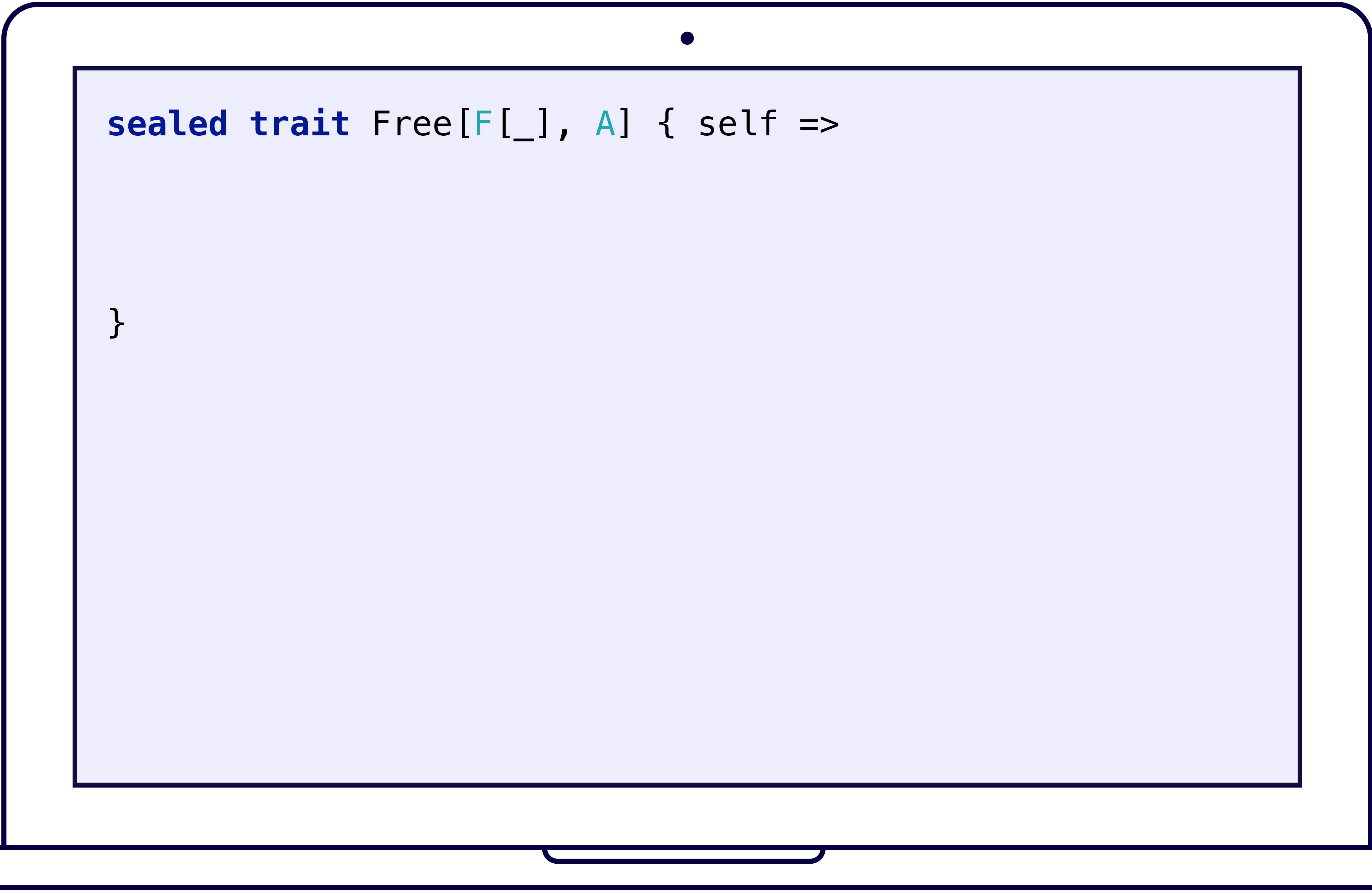
```
object IntegerAddition extends Monoid[Int] {  
  def append(a: Int, b: Int): Int = a + b  
  def identity: Int = 0  
}
```


Free Monads

Don't lose any data!
(that means no evaluating functions)

flatMap



A stylized illustration of a laptop with a light blue screen. The screen displays a snippet of Scala code. The code is:

```
sealed trait Free[F[_], A] { self =>  
  
}
```

 The words 'sealed' and 'trait' are in dark blue, 'Free' is in black, and 'F' and 'A' are in a light teal color. The rest of the code is in black. The laptop has a dark blue outline and a small black dot at the top center of the screen.

```
sealed trait Free[F[_], A] { self =>  
  
}
```

```
sealed trait Free[F[_], A] { self =>
```

```
}
```

```
case class Return[F[_], A](given: A) extends Free[F, A]
```


sealed trait Free[F[_], A] { self =>

}

case class Return[F[_], A](given: A) extends Free[F, A]

case class Suspend[F[_], A](fn: F[A]) extends Free[F, A]

```
sealed trait Free[F[_], A] { self =>
```

```
}
```

```
case class Return[F[_], A](given: A) extends Free[F, A]
```

```
case class Suspend[F[_], A](fn: F[A]) extends Free[F, A]
```

```
case class FlatMap[F[_], A, B]  
  (free: Free[F, A], fn: A => Free[F, B])  
  extends Free[F, B]
```

```
sealed trait Free[F[_], A] { self =>
  def flatMap ...
  def pure ...
  def map ...
}

case class Return[F[_], A](given: A) extends Free[F, A]

case class Suspend[F[_], A](fn: F[A]) extends Free[F, A]

case class FlatMap[F[_], A, B]
  (free: Free[F, A], fn: A => Free[F, B])
  extends Free[F, B]
```



```
sealed trait Todo[A]
case class NewTask[A](task: A) extends Todo[A]
case class CompleteTask[A](task: A) extends Todo[A]
case class GetTasks[A](default: A) extends Todo[A]
```

```
def newTask[A](task: A): Free[Todo, A] =  
  Suspend(NewTask(task))
```

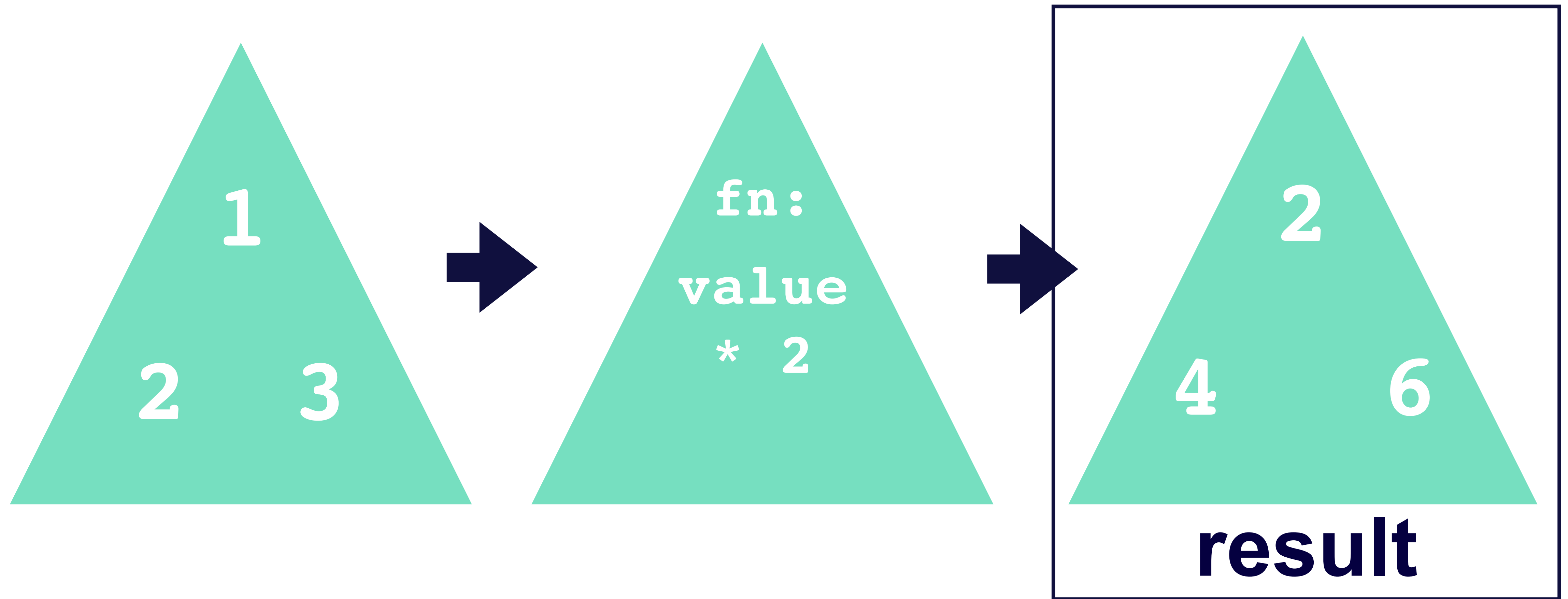
```
def completeTask[A](task: A): Free[Todo, A] =  
  Suspend(CompleteTask(task))
```

```
def getTasks[A](default: A): Free[Todo, A] =  
  Suspend(GetTasks(default))
```

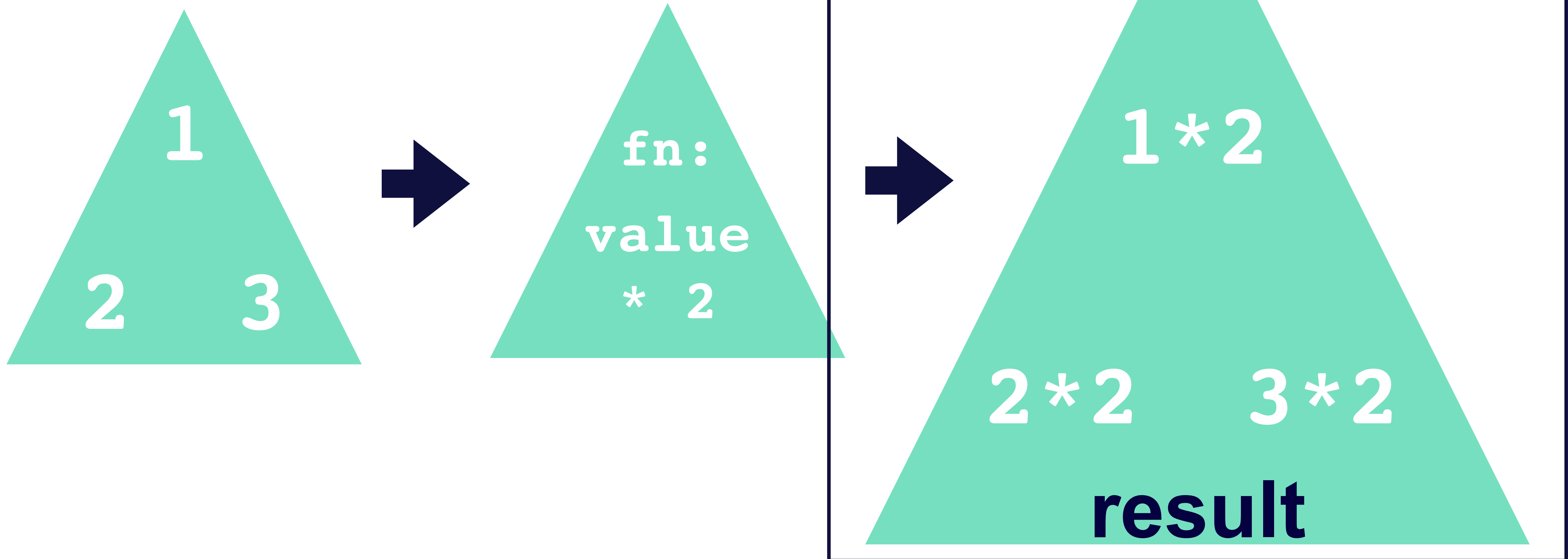
```
val todos: Free[Todo, Map[String, Boolean]] =  
  for {  
    _ <- newTask("Go to Curry On")  
    _ <- newTask("Write a novel")  
    _ <- newTask("Meet Tina Fey")  
    _ <- completeTask("Go to scala days")  
    tsks <- getTasks(Map.empty)  
  } yield tsks
```

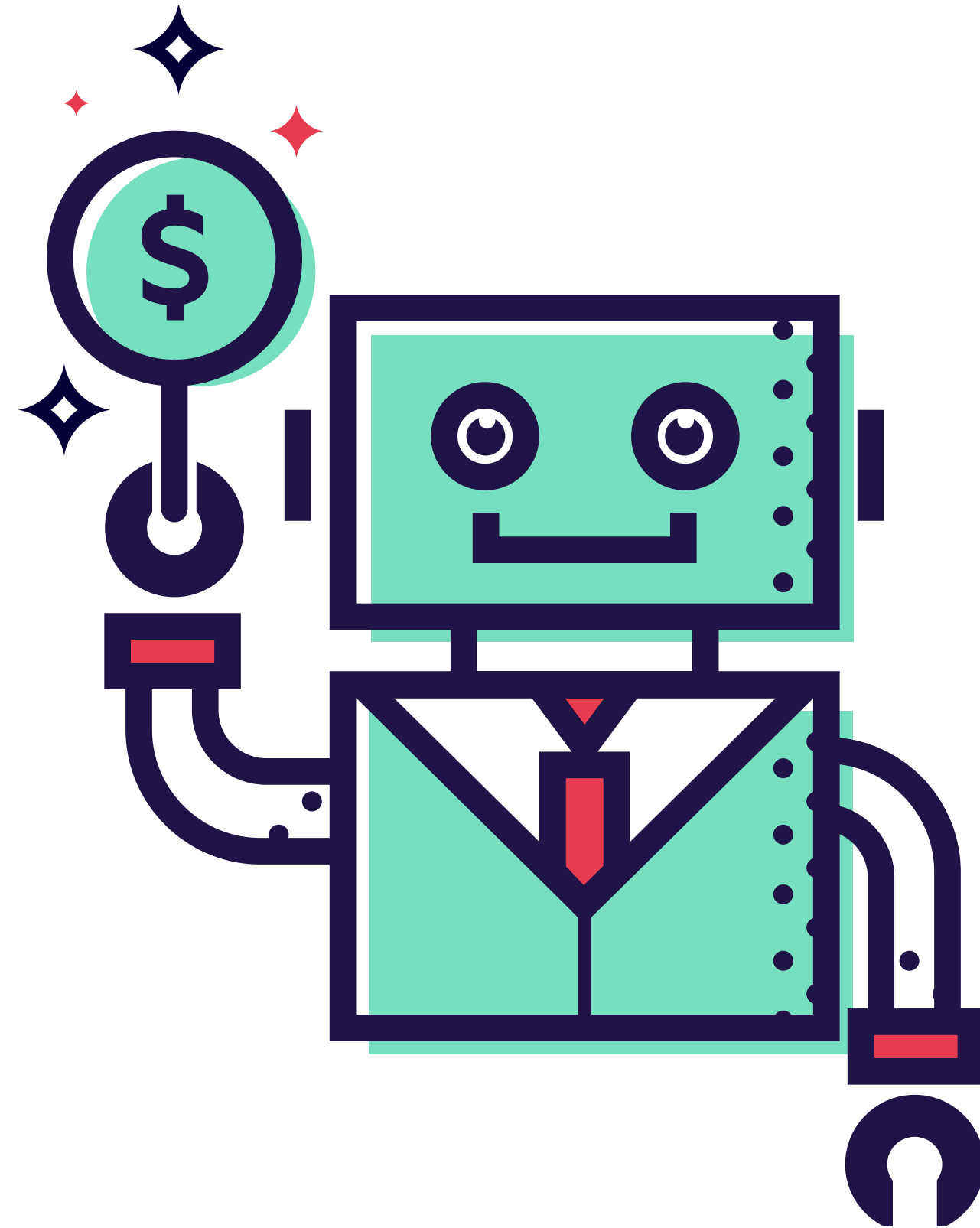
```
val todosExpanded: Free[Todo, Map[String, Boolean]] =  
  FlatMap(  
    Suspend(NewTask("Go to Curry On")), (a: String) =>  
    FlatMap(  
      Suspend(NewTask("Write a novel")), (b: String) =>  
      FlatMap(  
        Suspend(NewTask("Meet Tina Fey")), (c: String) =>  
        FlatMap(  
          Suspend(CompleteTask("Go to scala days")),  
          (d: String) =>  
            Suspend(GetTasks(default = Map.empty))  
        )  
      )  
    )  
  )  
)
```


I'm not free :(

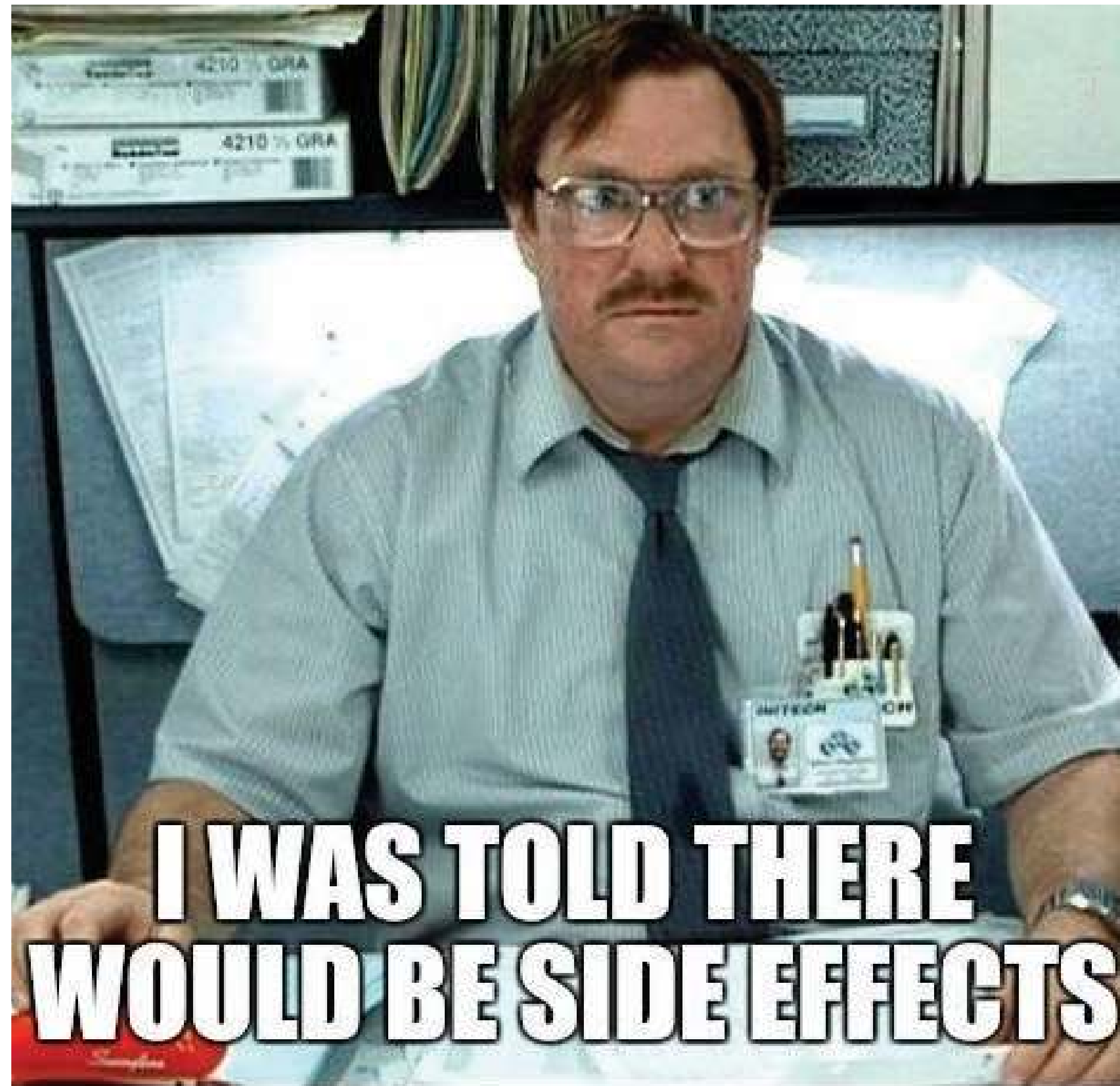


I'm free!





- Monoids, Functors & Monads
- How to be “Free”
- Why & Why Not “Free”
- Alternatives
- Real World Applications



@kelleyrobinson

What's the point?

- Defer side effects
- Multiple interpreters
- Stack safety

```
(1 to 1000).flatMap { i =>  
  doSomething(i).flatMap { j =>  
    doSomethingElse(j).flatMap { k =>  
      doAnotherThing(k).map { l =>  
        ...  
      }
```

**“Let’s just trampoline it and
add the Free Monad”**



Trampolining

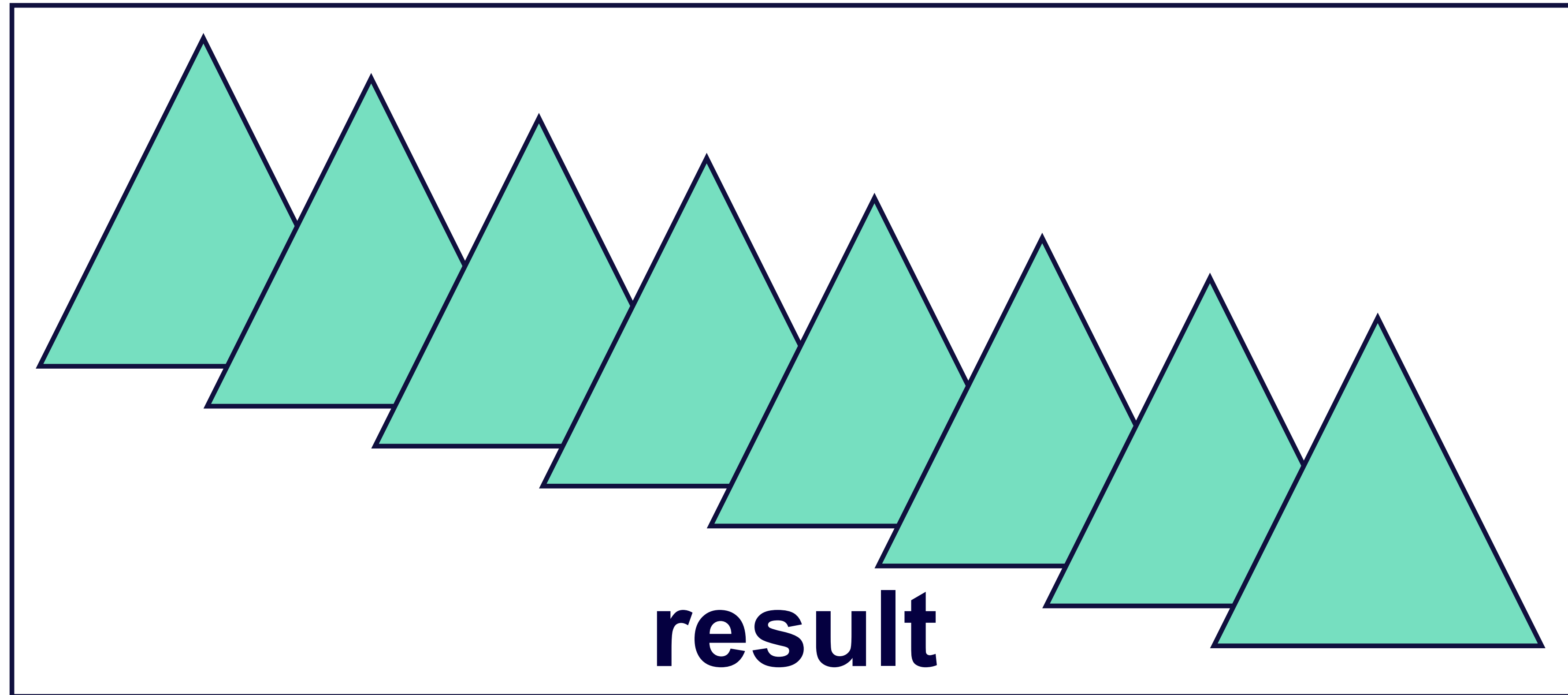
Express it in a loop

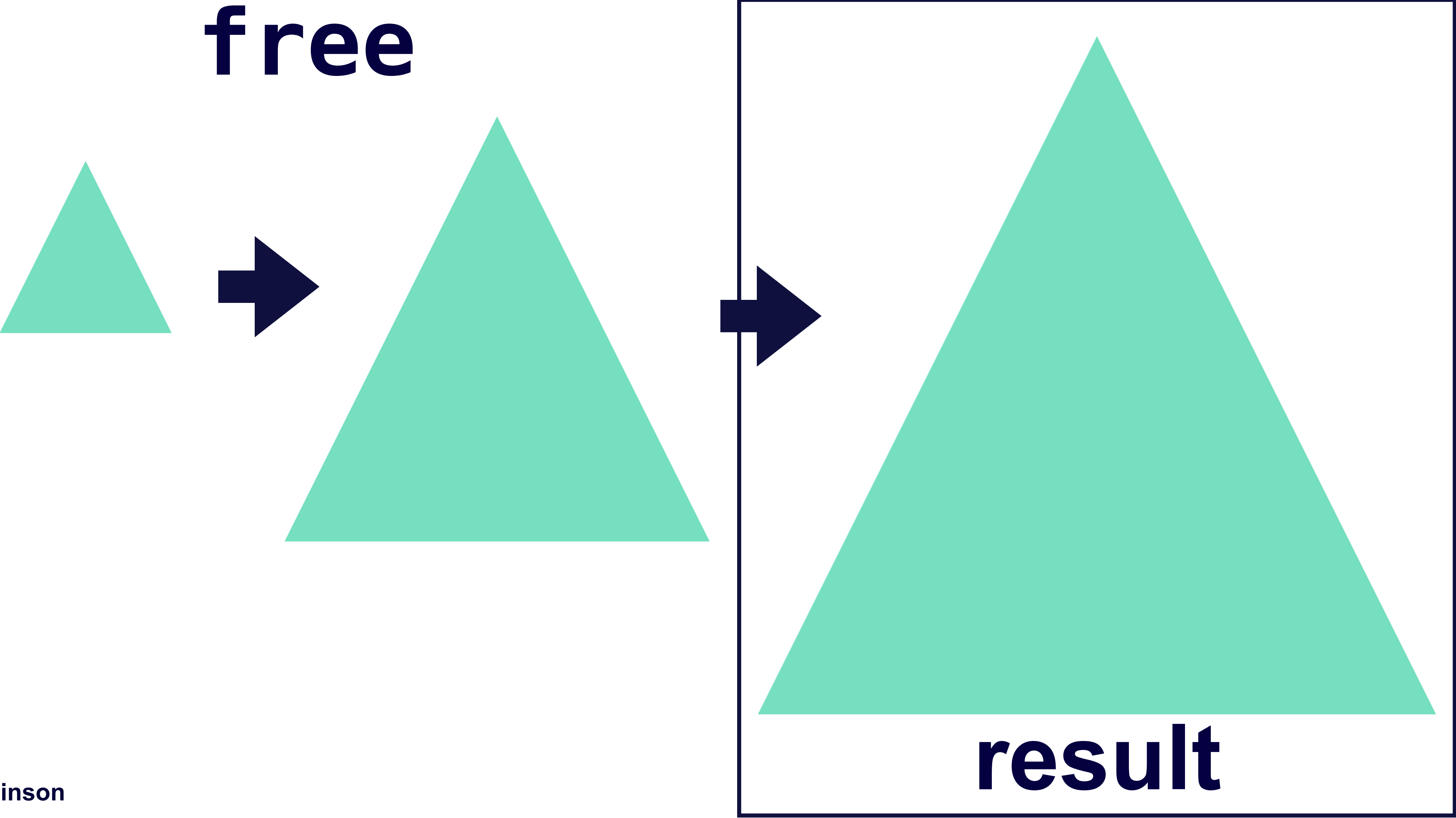
@kelleyrobinson

The Free Monad
uses **heap** instead of
using **stack**.



not free






```
val todosExpanded: Free[Todo, Map[String, Boolean]] =  
  FlatMap(  
    Suspend(NewTask("Go to Curry On")), (a: String) =>  
    FlatMap(  
      Suspend(NewTask("Write a novel")), (b: String) =>  
      FlatMap(  
        Suspend(NewTask("Meet Tina Fey")), (c: String) =>  
        FlatMap(  
          Suspend(CompleteTask("Go to scala days")),  
          (d: String) =>  
            Suspend(GetTasks(default = Map.empty))  
        )  
      )  
    )  
  )  
)
```


Evaluating

Use a loop

@kelleyrobinson


```
def runFree[F[_], G[_], A]  
  (f: Free[F, A])  
  (transform: ContextTransformer[F, G])  
  (implicit G: Monad[G]): G[A]
```

•
Turn F into G -

Input

AKA "Natural Transformation"

```
def runFree[F[_], G[_], A]  
  (f: Free[F, A])  
  (transform: FunctorTransformer[F, G])  
  (implicit G: Monad[G]): G[A]
```

`G` must be a monad so we can flatMap

```
// or 'NaturalTransformation'  
trait FunctorTransformer[F[_], G[_]] {  
  def apply[A](f: F[A]): G[A]  
}
```



```

/* Function body */

@annotation.tailrec
def tailThis(free: Free[F, A]): Free[F, A] = free match {
  case FlatMap(FlatMap(fr, fn1), fn2) => ...
  case FlatMap(Return(a), fn)         => ...
  case _                             => ...
}

tailThis(f) match {
  case Return(a)           => ...
  case Suspend(fa)         => ...
  case FlatMap(Suspend(fa), fn) => ...
  case _                   => ...
}

```

```
tailThis(f) match {  
  case Return(a)           => ...  
  case Suspend(fa)         => transform(fa)  
  case FlatMap(Suspend(fa), fn) =>  
    ... transform(fa) ...  
  case _                     => ...  
}
```



```

def runLoop(...) = {

  var eval: Free[F, A] = f

  while (true) {
    eval match {
      case Return(a)                => ...
      case Suspend(fa)              => ...
      case FlatMap(Suspend(fa), fn)  => ...
      case FlatMap(FlatMap(given, fn1), fn2) => ...
      case FlatMap(Return(s), fn)    => ...
    }
  }

  throw new AssertionError("Unreachable")
}

```


Evaluating

Applies transformation on
`Suspend`

Trampolining for stack safety


```
trait FunctorTransformer[F[_], G[_]] {  
  def apply[A](f: F[A]): G[A]  
}
```

```
type Id[A] = A

case class TestInterpreter(var model: Map[String, Boolean])
extends FunctorTransformer[Todo, Id] {

  def apply[A](a: Todo[A]): Id[A]

}
```

```
a match {  
  case NewTask(task) =>  
    model = model + (task.toString -> false)  
    task  
  case CompleteTask(task) =>  
    model = model + (task.toString -> true)  
    task  
  case GetTasks(default) =>  
    model.asInstanceOf[A]  
}
```



```
it("should evaluate todos") {  
  val result =  
    runFree(todos)(TestInterpreter(Map.empty))  
  
  val expected: Map[String, Boolean] =  
    Map(  
      "Go to Curry On" -> true,  
      "Write a novel" -> false,  
      "Meet Tina Fey" -> false  
    )  
  result shouldBe expected  
}
```

```
case object ActionTestInterpreter
extends FunctorTransformer[Todo, Id] {

  var actions: List[Todo[String]] = List.empty

  def apply[A](a: Todo[A]): Id[A]

}
```

```
a match {  
  case NewTask(task) =>  
    actions = actions :+ NewTask(task.toString)  
    task  
  case CompleteTask(task) =>  
    actions = actions :+ CompleteTask(task.toString)  
    task  
  case GetTasks(default) =>  
    actions = actions :+ GetTasks('')  
    default  
}
```



```
it("should evaluate todos actions in order") {  
  runFree(todos)(ActionTestInterpreter)  
  
  val expected: List[Todo[String]] =  
    List(  
      NewTask("Go to Curry On"),  
      NewTask("Write a novel"),  
      NewTask("Meet Tina Fey"),  
      CompleteTask("Go to scala days"),  
      GetTasks("")  
    )  
  
  ActionTestInterpreter.actions shouldBe expected  
}
```

**Defining multiple interpreters allows
you to test side-effecting code without
using testing mocks.**

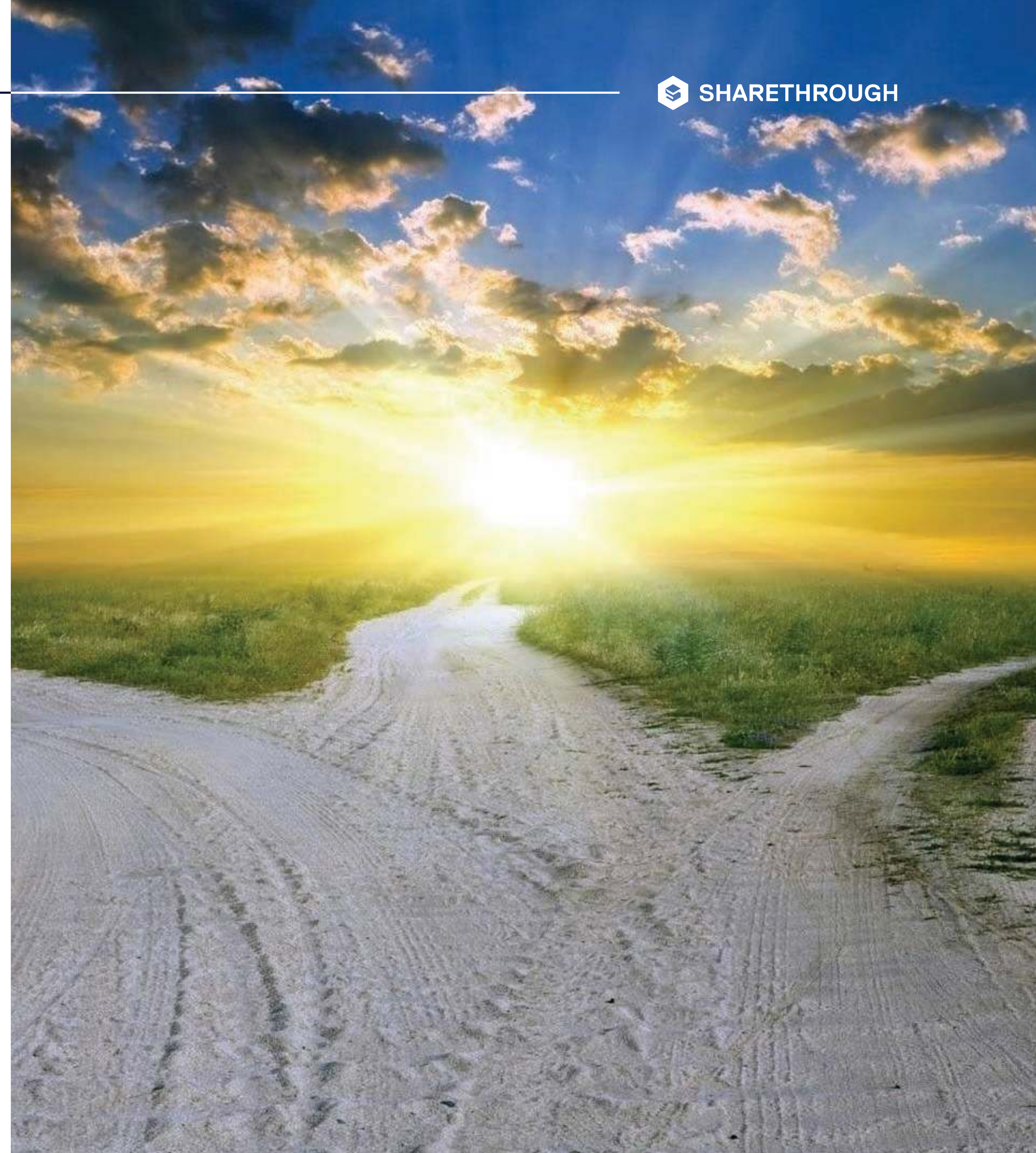
// Production Interpreter

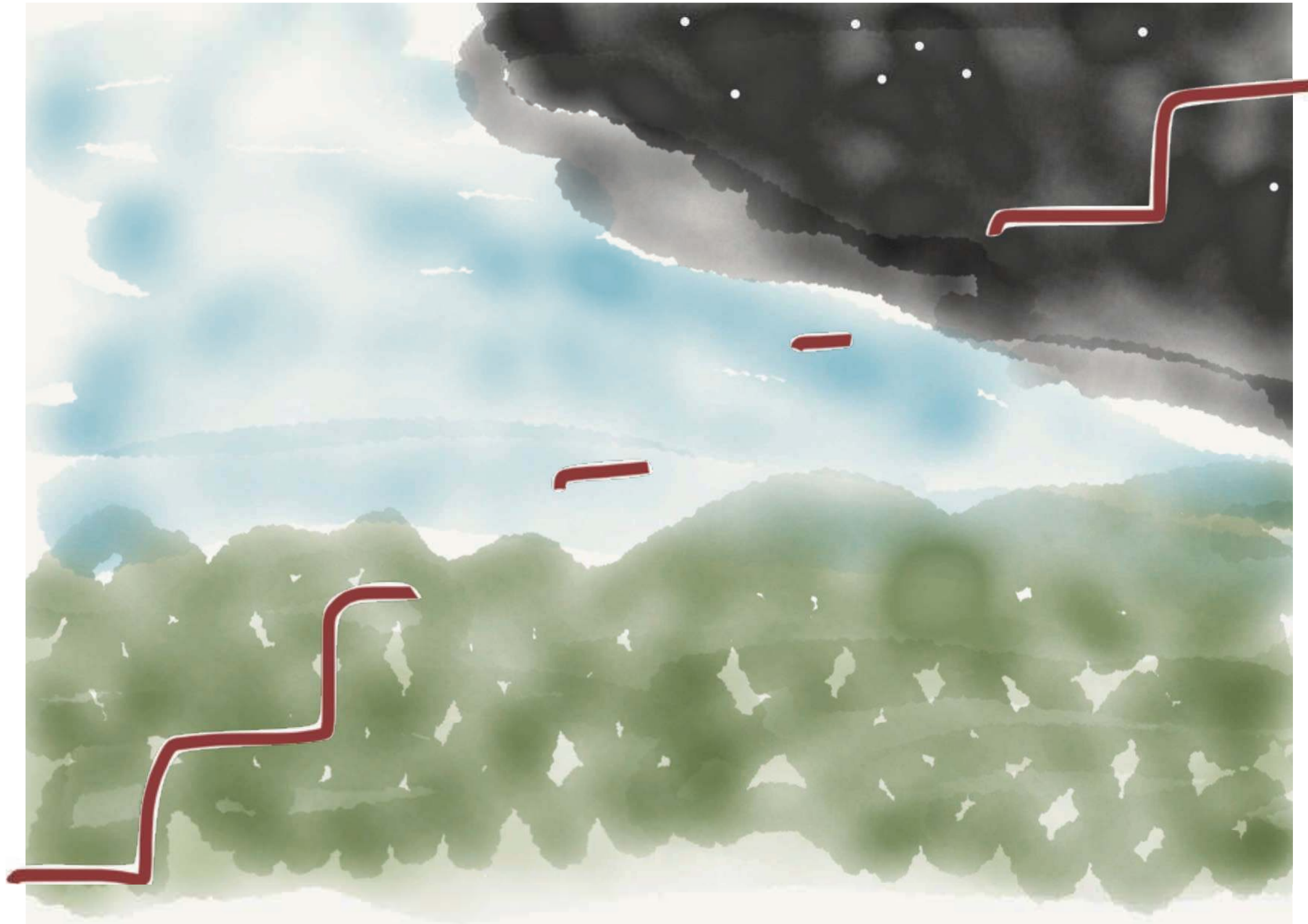
```
def apply[A](a: Todo[A]): Option[A] = {  
  a match {  
    case NewTask(task) =>  
      /**  
        * Some if DB write succeeds  
        * None if DB write fails  
        *  
        */  
    case CompleteTask(task) => ...  
    case GetTasks(default) => ...  
  }  
}
```


Justifications

- Defer side effects
- Multiple interpreters
- Stack safety

@kelleyrobinson





Credit: Jessica Kerr

#BlueSkyScala

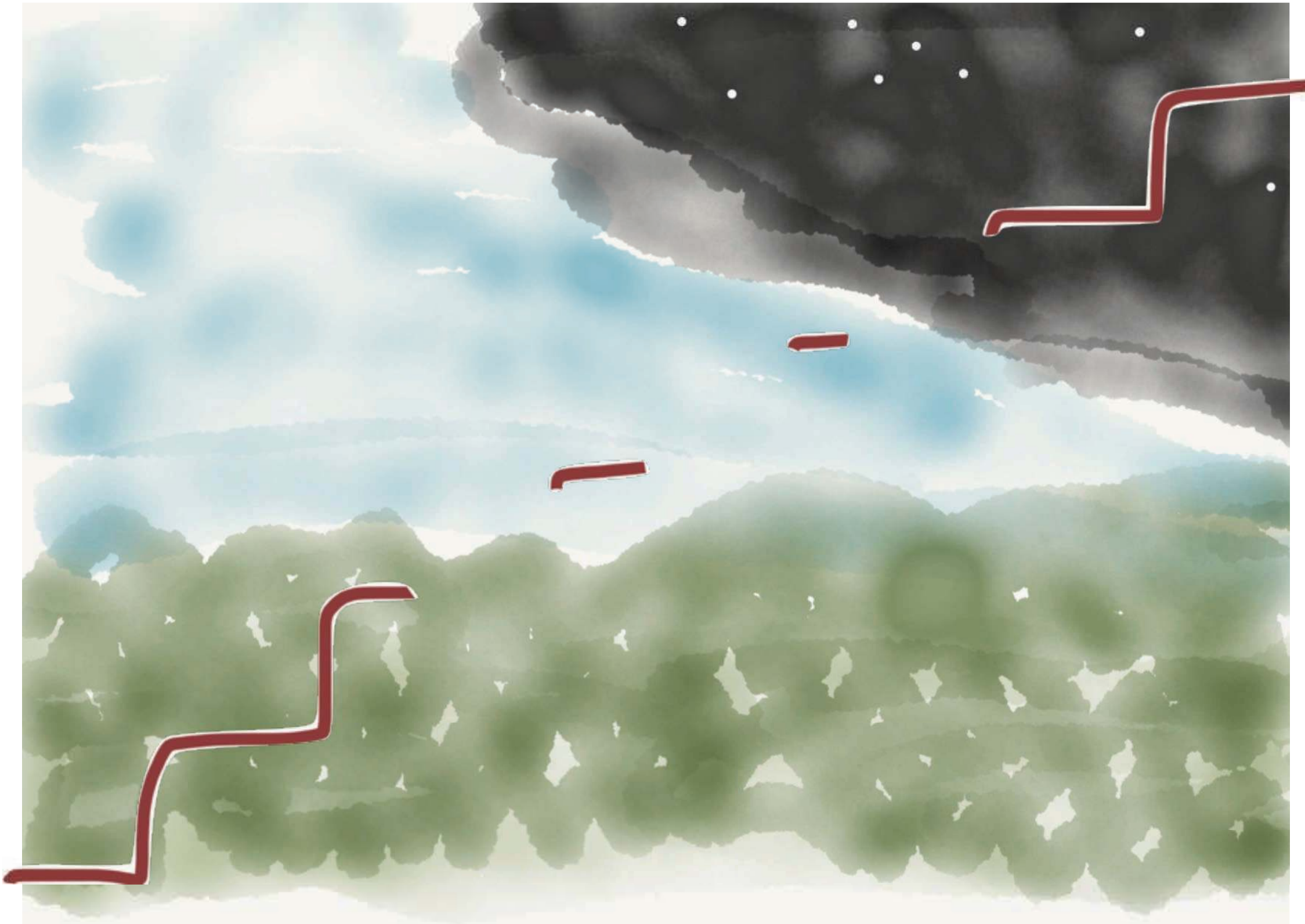
The path to learning is broken

@kelleyrobinson

*"Programming isn't math,
and sometimes we hoist vocabulary
that confers similar meanings"*

- Marius Eriksen



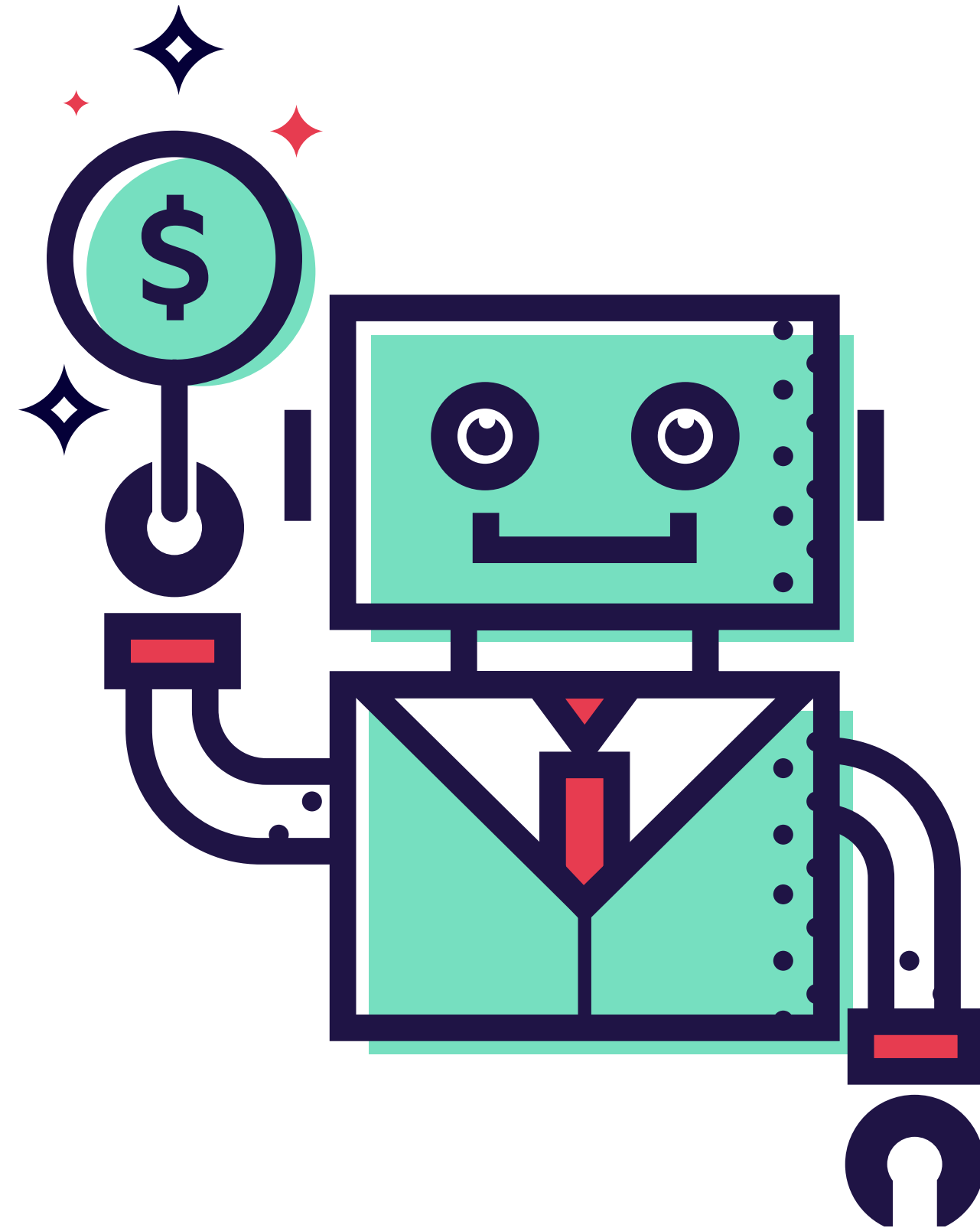


Credit: Jessica Kerr

Freedom isn't free

Reasons to avoid the Free Monad

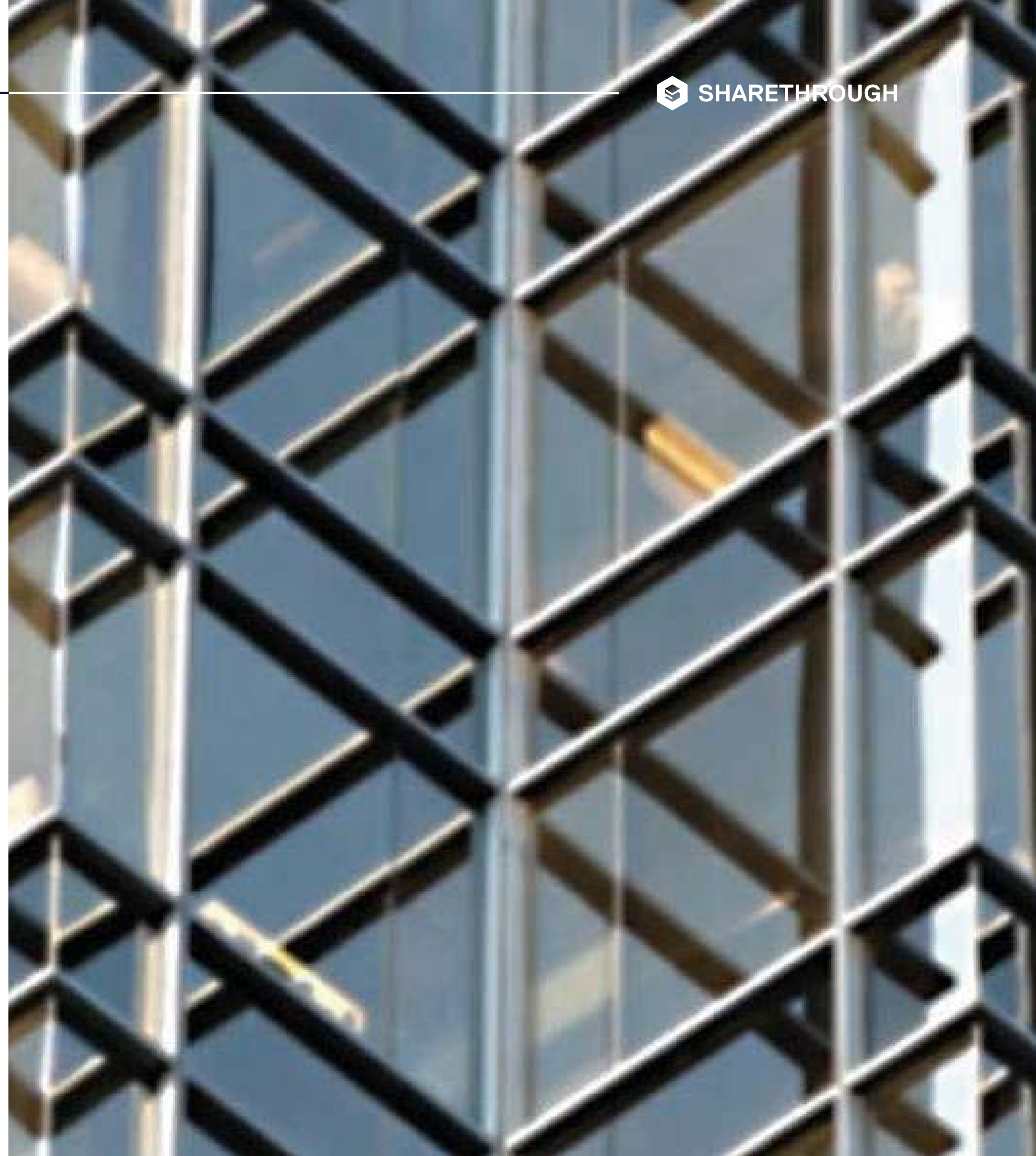
- Boilerplate
- Learning curve
- Alternatives



- Monoids, Functors & Monads
- How to be “Free”
- Why & Why Not “Free”
- Alternatives
- Real World Applications

Know your domain

@kelleyrobinson



Functional Spectrum

Where does your team fall?



Java



Haskell

Functional Spectrum

Where does your team fall?



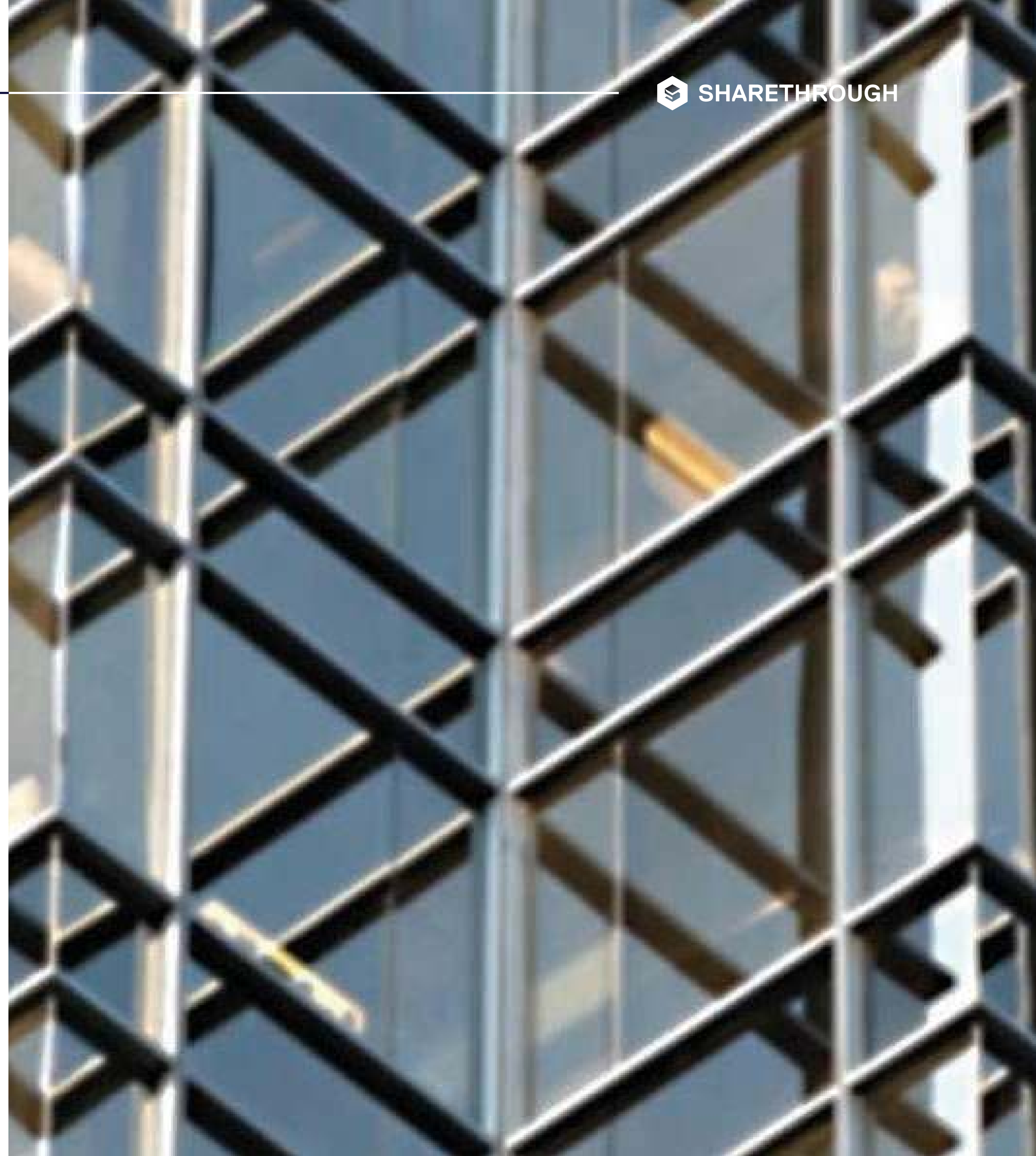
Java



Haskell

Alternatives for maintaining stack safety

@kelleyrobinson



```
final override def map[B, That](f: A => B)
  (implicit bf: CanBuildFrom[List[A], B, That]): That = {
```



Rúnar

@runarorama



Follow

LiterallyCantEvenBuildFrom

3:50 PM - 14 Apr 2016



18

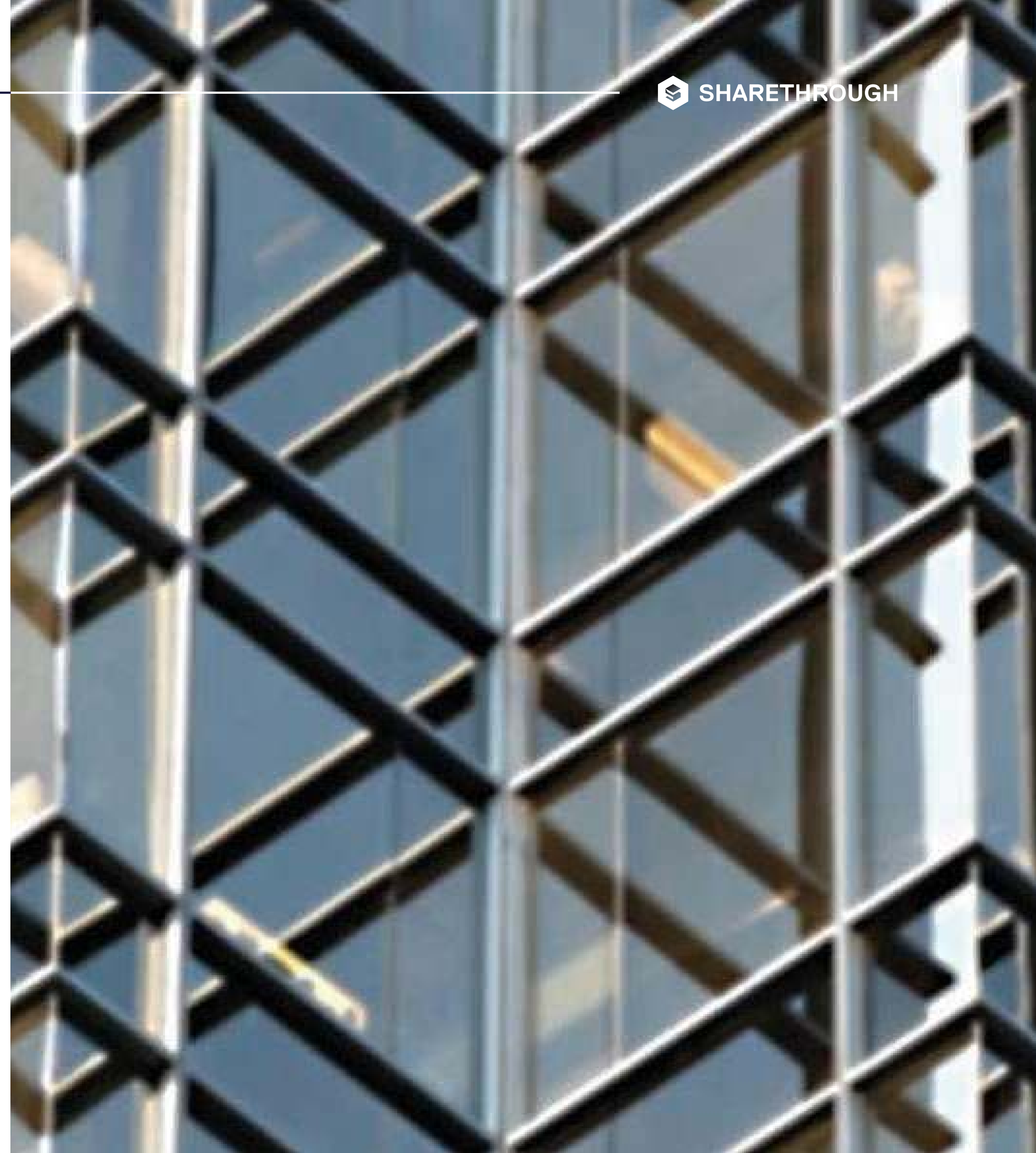


41

```
}
```

Alternatives for managing side effects

@kelleyrobinson

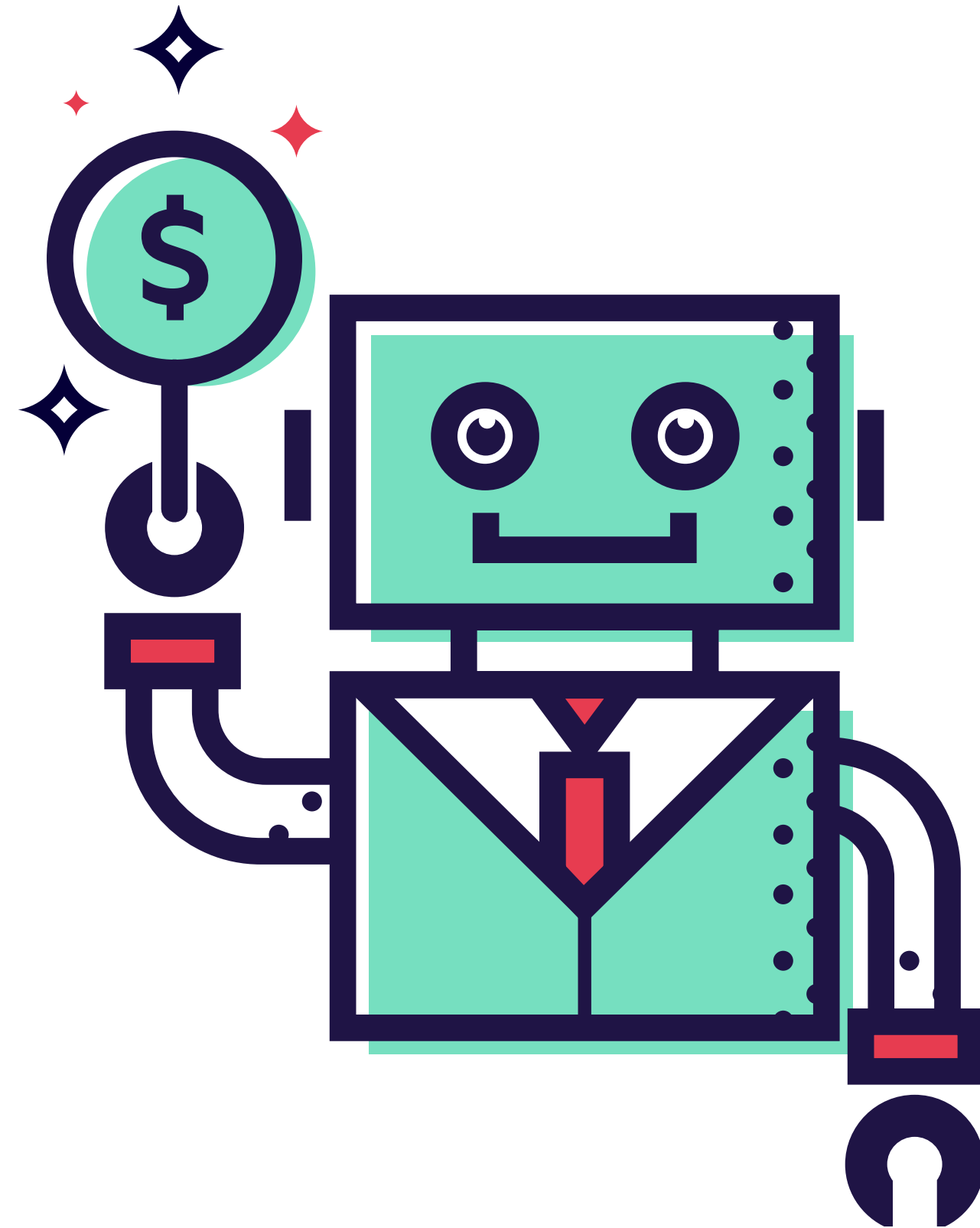



```
import java.sql.ResultSet

case class Person(name: String, age: Int)

def getPerson(rs: ResultSet): Person = {
  val name = rs.getString(1)
  val age = rs.getInt(2)
  Person(name, age)
}
```

```
def handleFailure[A](f: => A): ActionResult \/ A = {  
  Try(f) match {  
    case Success(res) => res.right  
    case Failure(e) =>  
      InternalServerError(reason = e.getMessage).left  
  }  
}  
  
handleFailure(getPerson(rs))
```

- Monoids, Functors & Monads
- How to be “Free”
- Why & Why Not “Free”
- Alternatives
- Real World Applications



Built in?

Some languages (Haskell) have built-in support

<https://wiki.haskell.org/Monad>



@kelleyrobinson

Language libs i.e. Scala Cats

Lightweight, modular, and extensible library for functional programming.

<http://typelevel.org/cats/>



Examples

- **Doobie**
- **scalaz.concurrent.Task**

<https://github.com/tpolecat/doobie>

@kelleyrobinson

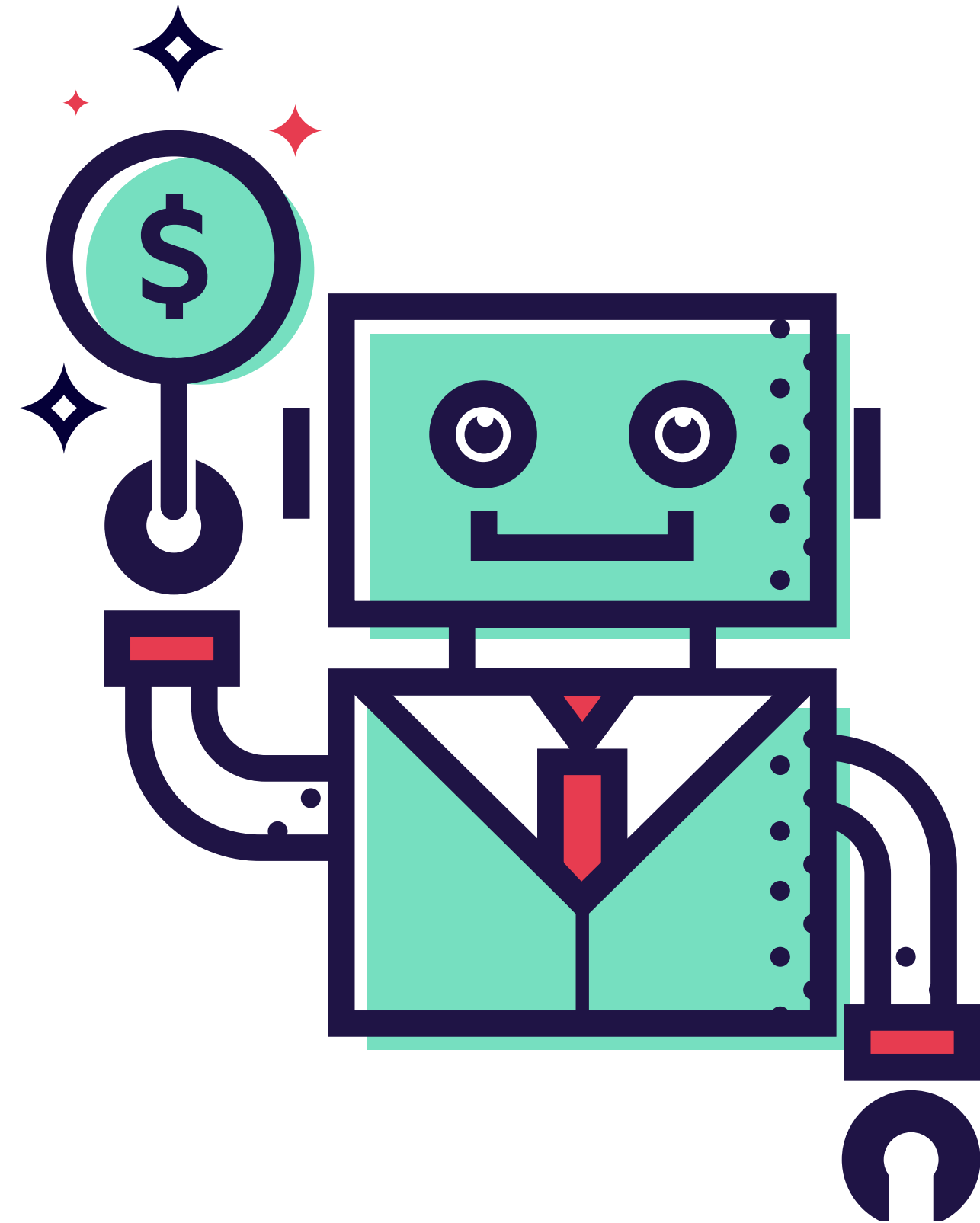

```
import scalaz.concurrent.Task

val tasks = messages.map(m => Task {
  processSQSMessage(conf, m)
})

Task.gatherUnordered(tasks).attemptRun match {
  case -\/(exp) => error(s"Failed processing")
  case _        => ()
}
```

```
// yikes
object Task {
  implicit val taskInstance:
    Nondeterminism[Task] with Catchable[Task]
    with MonadError[({type  $\lambda[\alpha, \beta]$  =
Task[ $\beta$ ]})# $\lambda$ , Throwable] = new
    Nondeterminism[Task] with Catchable[Task]
    with MonadError[({type  $\lambda[\alpha, \beta]$  =
Task[ $\beta$ ]})# $\lambda$ , Throwable] { ... }
}
```


My experience... ...what happened?



- **Know your domain**
- **Use clean abstractions**
- **Share knowledge**

Thank You!

@kelleyrobinson

hello@krobinson.me



Acknowledgements & Resources

Special thanks to:

- Sharethrough
- Rúnar Bjarnason
- Rob Norris
- Eugene Yokota
- Jessica Kerr
- David Hoyt
- Danielle Sucher
- Charles Ruhland

Resources for learning more about Free Monads:

- <http://blog.higher-order.com/assets/trampolines.pdf>
- <http://eed3si9n.com/learning-scalaz/>
- <https://stackoverflow.com/questions/44965/what-is-a-monad>
- <https://byorgey.wordpress.com/2009/01/12/abstraction-intuition-and-the-monad-tutorial-fallacy/>
- <http://hseeberger.github.io/blog/2010/11/25/introduction-to-category-theory-in-scala/>
- https://en.wikipedia.org/wiki/Free_object
- <https://softwaremill.com/free-monads/>
- <https://github.com/davidhoyt/kool-aid/>
- <https://www.youtube.com/watch?v=T4956GI-6Lw>

Other links and resources:

- <https://skillsmatter.com/skillscasts/6483-keynote-scaling-intelligence-moving-ideas-forward>
- <https://stackoverflow.com/questions/7213676/forall-in-scala-but-that-boilerplate>